

Data Cleaning

```
# Step 7: read csv file
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    4.0.0      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(janitor)
```

```
##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##   chisq.test, fisher.test
```

```
library(skimr)
library(lubridate) # for date changing

file_path <- "data/Airbnb_Open_Data_cleaned_first.csv"

df_cleaned_first <- read_csv(file_path) %>%
  clean_names()
```

```
## Rows: 102058 Columns: 23
## -- Column specification -----
## Delimiter: ","
## chr  (9): NAME, host_identity_verified, host name, neighbourhood group, neig...
## dbl  (13): id, host id, lat, long, Construction year, price, service fee, min...
## lgl  (1): instant_bookable
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
head(df_cleaned_first)
```

```
## # A tibble: 6 x 23
##       id name      host_id host_identity_verified host_name neighbourhood_group
##   <dbl> <chr>      <dbl> <chr>                <chr>      <chr>
## 1 1001254 Clean & ~ 8.00e10 unconfirmed          xiayi      Brooklyn
## 2 1002102 Skylit M~ 5.23e10 verified           Jenna      Manhattan
## 3 1002403 THE VILL~ 7.88e10 <NA>             Elise      Manhattan
## 4 1002755 blank     8.51e10 unconfirmed          Garry      Brooklyn
## 5 1003689 Entire A~ 9.20e10 verified           Lyndon     Manhattan
## 6 1004098 Large Co~ 4.55e10 verified           Michelle   Manhattan
## # i 17 more variables: neighbourhood <chr>, lat <dbl>, long <dbl>,
## #   instant_bookable <lgl>, cancellation_policy <chr>, room_type <chr>,
## #   construction_year <dbl>, price <dbl>, service_fee <dbl>,
## #   minimum_nights <dbl>, number_of_reviews <dbl>, last_review <chr>,
## #   reviews_per_month <dbl>, review_rate_number <dbl>,
## #   calculated_host_listings_count <dbl>, availability_365 <dbl>,
## #   house_rules <chr>
```

```
# Step 8 & 9: type transform and missing value
```

```
# median
```

```
median_reviews_per_month <- df_cleaned_first %>%
  pull(reviews_per_month) %>%
  median(na.rm = TRUE)
```

```
median_availability <- df_cleaned_first %>%
  pull(availability_365) %>%
  median(na.rm = TRUE)
```

```
# operation
```

```
df_cleaned_second <- df_cleaned_first %>%
  mutate(
    # mutate value type
    price = as.numeric(price),
    service_fee = as.numeric(service_fee),

    reviews_per_month = replace_na(reviews_per_month, median_reviews_per_month),

    availability_365 = replace_na(availability_365, median_availability),

  )
```

```
# check for missing value
```

```
skimr::skim(df_cleaned_second)
```

```
## Warning: There was 1 warning in `dplyr::summarize()`.
## i In argument: `dplyr::across(tidyselect::any_of(variable_names),
##   mangled_skimmers$funs)`.
## i In group 0: .
## Caused by warning:
## ! There were 3703 warnings in `dplyr::summarize()`.
## The first warning was:
## i In argument: `dplyr::across(tidyselect::any_of(variable_names),
##   mangled_skimmers$funs)`.
## Caused by warning in `grepl()`:
```

```
## ! unable to translate 'Williamsburg?<80>?Steps To Subway, Private Bath&Balcony' to a wide string
## i Run `dplyr::last_dplyr_warnings()` to see the 3702 remaining warnings.
```

Table 1: Data summary

Name	df_cleaned_second
Number of rows	102058
Number of columns	23
Column type frequency:	
character	9
logical	1
numeric	13
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
name	0	1.00	1	251	0	61250	0
host_identity_verified	289	1.00	8	11	0	2	0
host_name	0	1.00	1	35	0	13148	0
neighbourhood_group	29	1.00	5	13	0	5	0
neighbourhood	16	1.00	4	26	0	224	0
cancellation_policy	76	1.00	6	8	0	3	0
room_type	0	1.00	10	15	0	4	0
last_review	15832	0.84	8	10	0	2477	0
house_rules	51842	0.49	6	1004	0	1964	0

Variable type: logical

skim_variable	n_missing	complete_rate	mean	count
instant_bookable	105	1	0.5	FAL: 51186, TRU: 50767

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
id	0	1	2.918438e+07	1.627173e+07	1.007254e+07	1.509286e+07	2.918438e+07	4.327590e+07	5.736742e+07	
host_id	0	1	4.926738e+09	2.853742e+09	1.236005e+09	1.805992e+09	4.102865e+09	7.110061e+09	9.876313e+09	
lat	8	1	4.073000e+00	0.000000e+00	-40.50	4.069000e+00	4.072000e+00	4.076000e+00	4.092000e+01	
long	8	1	-74.25	5.000000e+01	-74.25	-	-	-	-	
construction_year	214	1	2.012490e+03	5.730000e+00	2.003.00	2.007000e+03	2.032000e+03	2.037000e+03	2.032000e+03	
price	247	1	6.253600e+02	3.326700e+01	50200	3.400000e+02	6.230000e+02	9.120000e+02	1.220000e+03	
service_fee	273	1	1.250400e+01	6.623000e+00	10100	6.800000e+00	1.250000e+01	1.620000e+01	2.420000e+02	
minimum_nights	400	1	8.150000e+00	3.034000e+00	1100	2.000000e+00	3.000000e+00	5.000000e+00	5.615000e+03	
number_of_reviews	183	1	2.752000e+01	4.957000e+00	10100	1.000000e+00	7.000000e+00	3.100000e+01	1.024000e+03	

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
reviews_per_month	0	1	1.280000e+00	1.020000e+00	0.001	2.800000e-01	7.400000e-01	1.710000e+00	9.000000e+01	
review_rate_number	319	1	3.280000e+00	2.000000e+00	0.000	2.000000e-01	3.000000e-01	4.000000e-01	5.000000e+00	
calculated_host_listings_count	1	1	7.940000e+00	3.027000e+01	0.000	1.000000e+00	1.000000e+00	2.000000e+00	3.320000e+02	
availability_365	0	1	1.408500e+03	1.331600e+02	0.000	3.000000e-01	9.600000e-01	2.680000e+00	3.677000e+03	

Step 10: change date

```
df_cleaned_second <- df_cleaned_second %>%
  mutate(
    last_review = lubridate::mdy(last_review) # use mdy()
  )

# define date limit
current_date_limit <- as.Date("2021-01-01")

# calculate median date
median_review_date <- df_cleaned_second %>%
  filter(!is.na(last_review) & last_review <= current_date_limit) %>%
  pull(last_review) %>%
  median(na.rm = TRUE)

# change
df_cleaned_second <- df_cleaned_second %>%
  mutate(
    # if last_review is NA or date exceeds limit, use median instead
    last_review = if_else(is.na(last_review) | last_review > current_date_limit,
                          median_review_date,
                          last_review)
  )

# check date
summary(df_cleaned_second$last_review)
```

```
##           Min.          1st Qu.          Median          Mean          3rd Qu.          Max.
## "2012-07-11" "2019-01-02" "2019-05-20" "2019-01-02" "2019-06-15" "2021-01-01"
```

Step 11: duplicate data handling

```
df_cleaned_second <- df_cleaned_second %>%
  # set minimum_nights
  mutate(
    minimum_nights = case_when(
      # no <0
      minimum_nights <= 0 ~ 1,
      # no outlier
      minimum_nights > 365 ~ 365,
      # everything else keep the same
      TRUE ~ minimum_nights
    )
  )
```

```
# check lines
print(paste("lines after duplicate data handling", nrow(df_cleaned_second)))
```

```
## [1] "lines after duplicate data handling 102058"
```

```
# check
summary(df_cleaned_second$minimum_nights)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
##    1.000   2.000   3.000   7.968   5.000 365.000    400
```